

	Departamento: Departamento de Ciencias de la Computación.
	Carrera: Software Asignatura: Análisis y Diseño de Software NRC: 30724
	Apellidos y nombres del estudiante: • Lasso Sebastián • Valencia Yuliana • Villarroel Justin Taller N°1

ANTECEDENTES

En el desarrollo de software existen diferentes formas de organizar un sistema para que sea más fácil de entender, mantener y mejorar. Una de estas formas es mediante el uso de patrones de diseño, los cuales ayudan a resolver problemas comunes de programación.

Dentro de estos patrones se encuentran los patrones estructurales, que sirven para organizar las relaciones entre clases y objetos de una mejor manera. En este trabajo se estudiarán los patrones Composite, Proxy y Bridge, ya que cada uno cumple una función diferente dentro de un sistema.

El patrón Composite ayuda a trabajar con objetos individuales y grupos de objetos de la misma manera. El patrón Proxy permite controlar el acceso a ciertas partes del sistema usando un intermediario. Finalmente, el patrón Bridge ayuda a separar una parte del sistema de su implementación para que sea más flexible y fácil de modificar.

Estos patrones pueden aplicarse en un sistema de matriculación estudiantil para mejorar la organización del código y facilitar procesos como el registro y administración de estudiantes.

OBJETIVO

Analizar e implementar los patrones de diseño estructurales Composite, Proxy y Bridge en el desarrollo de un sistema de matriculación estudiantil, con el fin de mejorar la organización, acceso y flexibilidad de la estructura del software.

DESARROLLO

1. Composite

Descripción:

El patrón Composite pertenece a los patrones de diseño estructurales, ya que se enfoca en la forma en que las clases y objetos se organizan dentro de un sistema para construir estructuras complejas.

Este patrón permite componer objetos en estructuras jerárquicas tipo árbol, de manera que los objetos individuales y los grupos de objetos puedan ser tratados de forma uniforme. En otras palabras, un objeto principal puede estar formado por varios objetos secundarios, manteniendo una misma interfaz de operación.

La principal ventaja del patrón Composite es que simplifica la gestión de estructuras complejas, ya que el sistema puede manipular tanto elementos simples como compuestos utilizando los mismos métodos.

Como funciona:

El patrón Composite funciona organizando objetos en forma de una estructura jerárquica, donde un objeto principal puede contener otros objetos. Esto permite trabajar de la misma manera con un solo elemento o con un grupo de elementos. Por ejemplo, en un sistema de matriculación, se puede manejar un estudiante individual o una lista de estudiantes usando las mismas operaciones del sistema.

@startuml

skinparam classAttributeIconSize 0

skinparam shadowing false

left to right direction

' --- VISTA Y CONTROLADOR ---

class "FormularioCrudEstudiante" <<boundary>> {

- txtId: String

- txtNombre: String

- txtEdad: String

+ clickAgregar()

+ clickActualizar()

+ clickEliminar()

+ clickMostrarTodo()

}

class "ControlEstudiante" <<control>> {

+ agregarEstudiante(id: String, nombre: String, edad: int): void

+ crearGrupo(idGrupo: String, nombreGrupo: String): void

+ agruparComponente(idGrupo: String, comp: IComponenteAcademico): void

+ eliminarComponente(id: String): void

+ mostrarTodos(): void

}

' --- PATRÓN COMPOSITE ---

interface "IComponenteAcademico" <<component>> {

+ getId(): String

+ mostrarDatos(): String

}

```

class "Estudiante" <<leaf>> {
  - id: String
  - nombre: String
  - edad: int
  + Estudiante(id: String, nombre: String, edad: int)
  + getId(): String
  + mostrarDatos(): String
}

```

```

class "GrupoEstudiantes" <<composite>> {
  - id: String
  - nombreGrupo: String
  - componentes: List<IComponenteAcademico>
  + GrupoEstudiantes(id: String, nombreGrupo: String)
  + agregar(c: IComponenteAcademico): void
  + remover(id: String): void
  + getId(): String
  + mostrarDatos(): String
}

```

' --- ALMACENAMIENTO (Sin Bridge/Proxy) ---

```

class "RepositorioMemoria" <<repository>> {
  - elementos: List<IComponenteAcademico>
  + existId(id: String): boolean
  + guardar(comp: IComponenteAcademico): void
  + buscarPorId(id: String): IComponenteAcademico
  + eliminar(id: String): void
  + listarTodos(): List<IComponenteAcademico>
}

```

' --- RELACIONES ---

FormularioCrudEstudiante --> ControlEstudiante : solicita
 ControlEstudiante --> RepositorioMemoria : interactúa
 ControlEstudiante ..> IComponenteAcademico : crea/gestiona

' Relaciones del Composite

IComponenteAcademico <|.. Estudiante

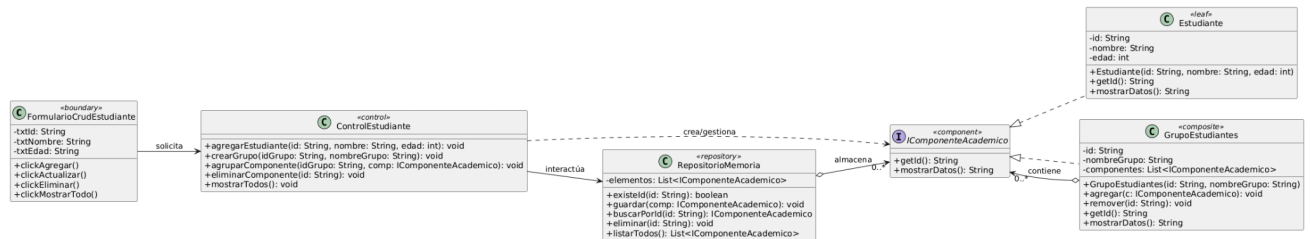
IComponenteAcademico <|.. GrupoEstudiantes

GrupoEstudiantes o--> "0..*" IComponenteAcademico : contiene

' Repositorio almacena la abstracción del composite

RepositorioMemoria o--> "0..*" IComponenteAcademico : almacena

@enduml



2. Proxy (Apoderado)

Descripción: Es un patrón de diseño estructural que proporciona un sustituto o marcador de posición (placeholder) para otro objeto con el fin de controlar el acceso a éste.

Cómo funciona: El Proxy implementa exactamente la misma interfaz que el objeto real al que representa (Sujeto Real).

Cuando el cliente hace una solicitud, la hace al Proxy creyendo que es el objeto original. El Proxy intercepta esta solicitud y realiza operaciones adicionales de gestión antes o después de pasar la solicitud al objeto real.

Plantuml Proxy:

@startuml

skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

```
class "FormularioCrudEstudiante" <<boundary>> {
- txtId: String
- txtNombre: String
- txtEdad: String
+ clickAgregar()
+ clickActualizar()
+ clickEliminar()
+ clickMostrarTodo()
+ mostrarMensaje(mensaje: String)
+ mostrarTabla(estudiantes: List<Estudiante>)
}
```

```
class "ControlEstudiante" <<control>> {
+ agregarEstudiante(id: String, nombre: String, edad: int): void
+ actualizarEstudiante(id: String, nombre: String, edad: int): void
+ eliminarEstudiante(id: String): void
}
```

```

+ mostrarTodos(): List<Estudiante>
}

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

interface "IServicioEstudiante" <<subject>> {
+ guardarEstudiante(est: Estudiante): boolean
+ modificarEstudiante(est: Estudiante): boolean
+ borrarEstudiante(id: String): boolean
+ listarEstudiantes(): List<Estudiante>
}

class "ServicioEstudianteReal" <<real subject>> {
- lista: List<Estudiante>
+ guardarEstudiante(est: Estudiante): boolean
+ modificarEstudiante(est: Estudiante): boolean
+ borrarEstudiante(id: String): boolean
+ listarEstudiantes(): List<Estudiante>
}

class "ProxyProxyEstudiante" <<proxy>> {
- servicioReal: ServicioEstudianteReal
- cacheEstudiantes: List<Estudiante>
+ guardarEstudiante(est: Estudiante): boolean
+ modificarEstudiante(est: Estudiante): boolean
+ borrarEstudiante(id: String): boolean
+ listarEstudiantes(): List<Estudiante>
- verificarAcceso(): boolean
- registrarLog(accion: String): void
}

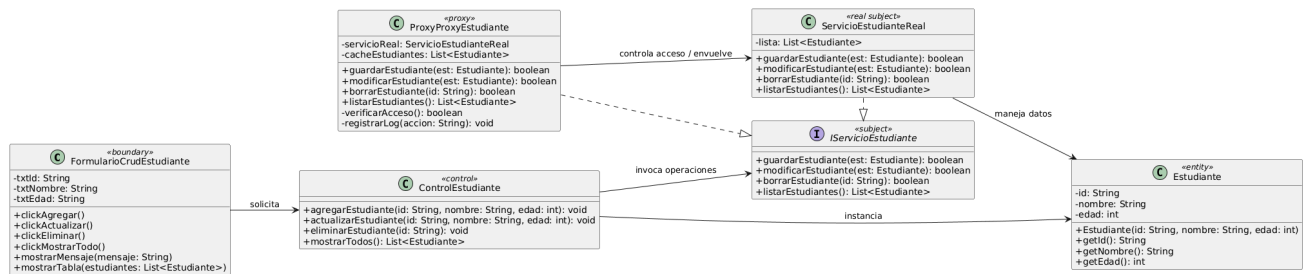
FormularioCrudEstudiante --> ControlEstudiante : solicita
ControlEstudiante --> IServicioEstudiante : invoca operaciones
ControlEstudiante --> Estudiante : instancia

ServicioEstudianteReal .|> IServicioEstudiante
ProxyProxyEstudiante .|> IServicioEstudiante

ProxyProxyEstudiante --> ServicioEstudianteReal : controla acceso / envuelve
ServicioEstudianteReal --> Estudiante : maneja datos

@enduml

```



3. Bridge (Puentes)

Descripción: Es un patrón de diseño estructural que desacopla una abstracción de su implementación, de modo que ambas puedan variar o evolucionar de forma completamente independiente.

Cómo funciona: Normalmente, si tienes una clase con muchas variantes (por ejemplo, Forma que puede ser Circulo o Cuadrado) y necesitas que funcione en múltiples plataformas (por ejemplo, Windows o Linux), podrías terminar usando herencia múltiple o creando una cantidad explosiva de subclases (CirculoWindows, CirculoLinux, CuadradoWindows, etc.). El patrón Bridge resuelve esto cambiando la herencia por composición.

Plantuml Bridge:

@startuml

skinparam classAttributeIconSize 0

skinparam shadowing false

left to right direction

```

class "FormularioCrudEstudiante" <<boundary>> {
    - txtId: String
    - txtNombre: String
    - txtEdad: String
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + mostrarMensaje(mensaje: String)
    + mostrarTabla(estudiantes: List<Estudiante>)
}
    
```

```

class "ControlEstudiante" <<control>> {
    + agregarEstudiante(id: String, nombre: String, edad: int): void
    + actualizarEstudiante(id: String, nombre: String, edad: int): void
}
    
```

```

+ eliminarEstudiante(id: String): void
+ mostrarTodos(): List<Estudiante>
}

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

abstract class "AbstractEstudiante" <<abstraction>> {
# implementador: ImplementadorEstudiante
+ AbstractEstudiante(imp: ImplementadorEstudiante)
+ {abstract} procesarAgregar(est: Estudiante): boolean
+ {abstract} procesarActualizar(est: Estudiante): boolean
+ {abstract} procesarEliminar(id: String): boolean
+ {abstract} obtenerTodos(): List<Estudiante>
}

class "GestorEstudiante" <<refined abstraction>> {
+ procesarAgregar(est: Estudiante): boolean
+ procesarActualizar(est: Estudiante): boolean
+ procesarEliminar(id: String): boolean
+ obtenerTodos(): List<Estudiante>
}

interface "ImplementadorEstudiante" <<implementor>> {
+ existe(id: String): boolean
+ guardarMemoria(est: Estudiante): void
+ modificarMemoria(est: Estudiante): void
+ borrarMemoria(id: String): void
+ listarMemoria(): List<Estudiante>
}

class "ImplementadorListaMemoria" <<concrete implementor>> {
- lista: List<Estudiante>

```

```

+ existe(id: String): boolean
+ guardarMemoria(est: Estudiante): void
+ modificarMemoria(est: Estudiante): void
+ borrarMemoria(id: String): void
+ listarMemoria(): List<Estudiante>
}

```

FormularioCrudEstudiante --> ControlEstudiante : solicita

ControlEstudiante --> AbstractEstudiante : delega validación CRUD

ControlEstudiante --> Estudiante : instancia

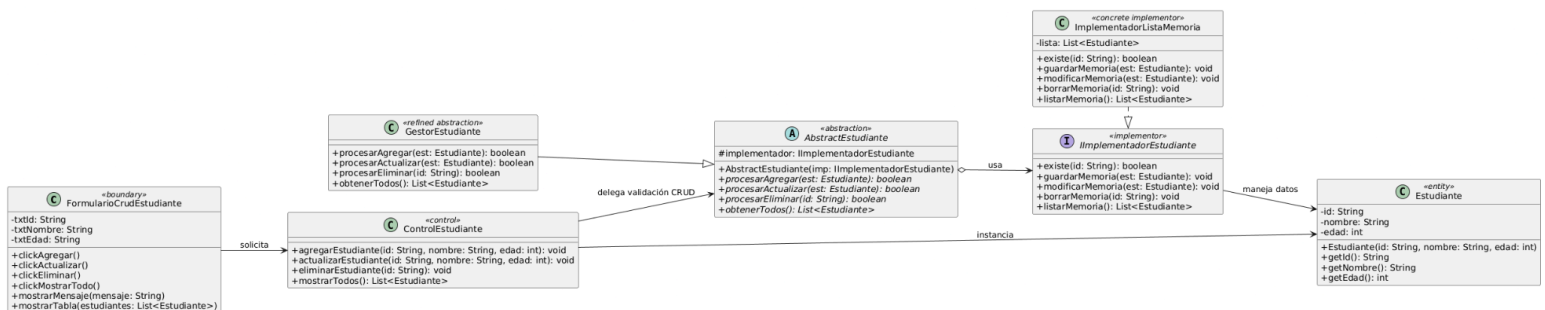
AbstractEstudiante o--> ImplementadorEstudiante : usa

GestorEstudiante --|> AbstractEstudiante

ImplementadorListaMemoria .|> ImplementadorEstudiante

ImplementadorEstudiante --> Estudiante : maneja datos

@enduml



4. Implementación de los 3 patrones de diseño:

Plantuml:

@startuml

skinparam classAttributeIconSize 0

skinparam shadowing false

left to right direction

' --- VISTA Y CONTROLADOR ---

```
class "FormularioCrudEstudiante" <<boundary>> {
  - txtId: String
  - txtNombre: String
  - txtEdad: String
  + clickAgregar()
  + clickActualizar()
  + clickEliminar()
  + clickMostrarTodo()
  + mostrarMensaje(mensaje: String)
  + mostrarTabla(estudiantes: List<IComponenteAcademico>)
}

class "ControlEstudiante" <<control>> {
  + agregarComponente(id: String, nombre: String, edad: int): void
  + crearGrupo(id: String, nombre: String): void
  + actualizarComponente(id: String, nombre: String, edad: int): void
  + eliminarComponente(id: String): void
  + mostrarTodos(): List<IComponenteAcademico>
}

' --- PATRÓN COMPOSITE ---

interface "IComponenteAcademico" <<component>> {
  + getId(): String
  + mostrarDatos(): String
}

class "Estudiante" <<leaf>> {
  - id: String
  - nombre: String
  - edad: int
  + Estudiante(id: String, nombre: String, edad: int)
  + getId(): String
  + getNombre(): String
  + getEdad(): int
  + mostrarDatos(): String
}
```

```

}

class "GrupoEstudiantes" <<composite>> {
  - id: String
  - nombreGrupo: String
  - componentes: List<IComponenteAcademico>
  + agregar(c: IComponenteAcademico): void
  + remover(id: String): void
  + getId(): String
  + mostrarDatos(): String
}

' --- PATRÓN PROXY ---

interface "IServicioEstudiante" <<subject>> {
  + guardar(comp: IComponenteAcademico): boolean
  + modificar(comp: IComponenteAcademico): boolean
  + borrar(id: String): boolean
  + listar(): List<IComponenteAcademico>
}

class "ProxyProxyEstudiante" <<proxy>> {
  - servicioReal: ServicioEstudianteReal
  - cacheComponentes: List<IComponenteAcademico>
  + guardar(comp: IComponenteAcademico): boolean
  + modificar(comp: IComponenteAcademico): boolean
  + borrar(id: String): boolean
  + listar(): List<IComponenteAcademico>
  - verificarAcceso(): boolean
  - registrarLog(accion: String): void
}

' --- PATRÓN BRIDGE ---

abstract class "AbstractServicio" <<abstraction>> {
  # implementador: IImplementadorMemoria
  + AbstractServicio(imp: IImplementadorMemoria)
  + {abstract} guardar(comp: IComponenteAcademico): boolean
  + {abstract} modificar(comp: IComponenteAcademico): boolean
  + {abstract} borrar(id: String): boolean
}

```

```

+ {abstract} listar(): List<IComponenteAcademico>
}

class "ServicioEstudianteReal" <<real subject / refined abs>> {
+ guardar(comp: IComponenteAcademico): boolean
+ modificar(comp: IComponenteAcademico): boolean
+ borrar(id: String): boolean
+ listar(): List<IComponenteAcademico>
}

interface "ImplementadorMemoria" <<implementor>> {
+ existe(id: String): boolean
+ guardarMemoria(comp: IComponenteAcademico): void
+ modificarMemoria(comp: IComponenteAcademico): void
+ borrarMemoria(id: String): void
+ listarMemoria(): List<IComponenteAcademico>
}

class "ImplementadorListaMemoria" <<concrete implementor>> {
- lista: List<IComponenteAcademico>
+ existe(id: String): boolean
+ guardarMemoria(comp: IComponenteAcademico): void
+ modificarMemoria(comp: IComponenteAcademico): void
+ borrarMemoria(id: String): void
+ listarMemoria(): List<IComponenteAcademico>
}

' --- RELACIONES ---

FormularioCrudEstudiante --> ControlEstudiante : solicita
ControlEstudiante --> IServicioEstudiante : invoca operaciones
ControlEstudiante ..> IComponenteAcademico : instancia

' Relaciones Composite
IComponenteAcademico <|.. Estudiante
IComponenteAcademico <|.. GrupoEstudiantes
GrupoEstudiantes o--> "0..*" IComponenteAcademico : contiene

' Relaciones Proxy
IServicioEstudiante <|.. ProxyProxyEstudiante

```

ProxyProxyEstudiante o--> ServicioEstudianteReal : envuelve / controla

' Relaciones Bridge (Conectando el Real Subject a la Abstracción)

IServicioEstudiante <|.. AbstractServicio

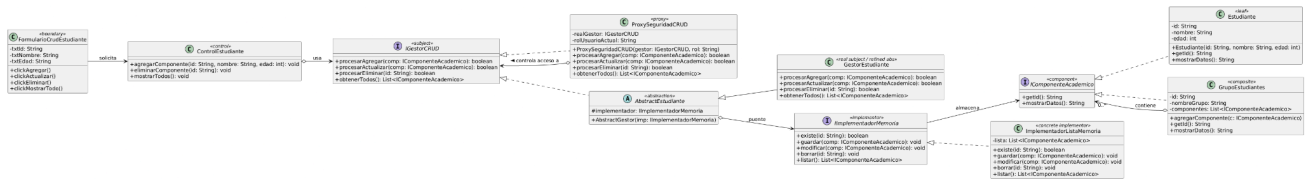
AbstractServicio <|-- ServicioEstudianteReal

AbstractServicio o--> ImplementadorMemoria : puente (bridge)

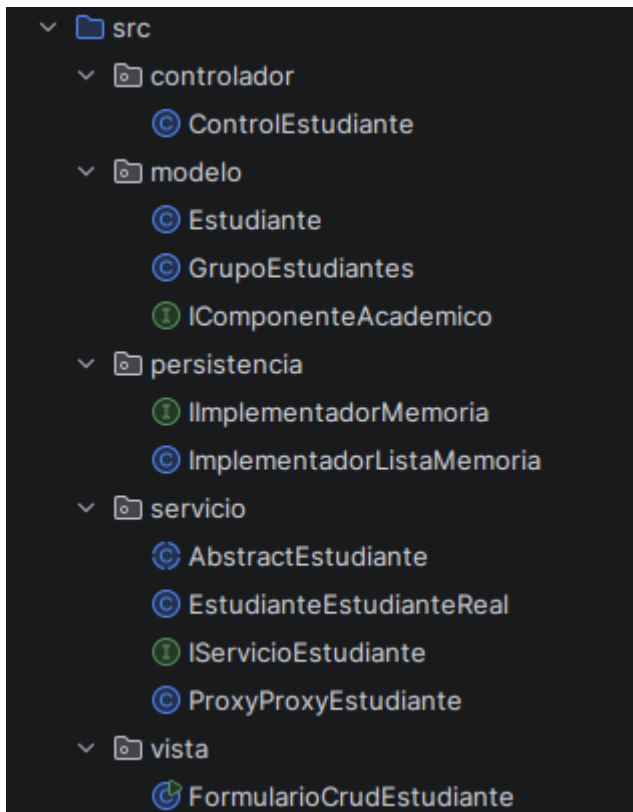
ImplementadorMemoria <|.. ImplementadorListaMemoria

ImplementadorMemoria --> IComponenteAcademico : manipula

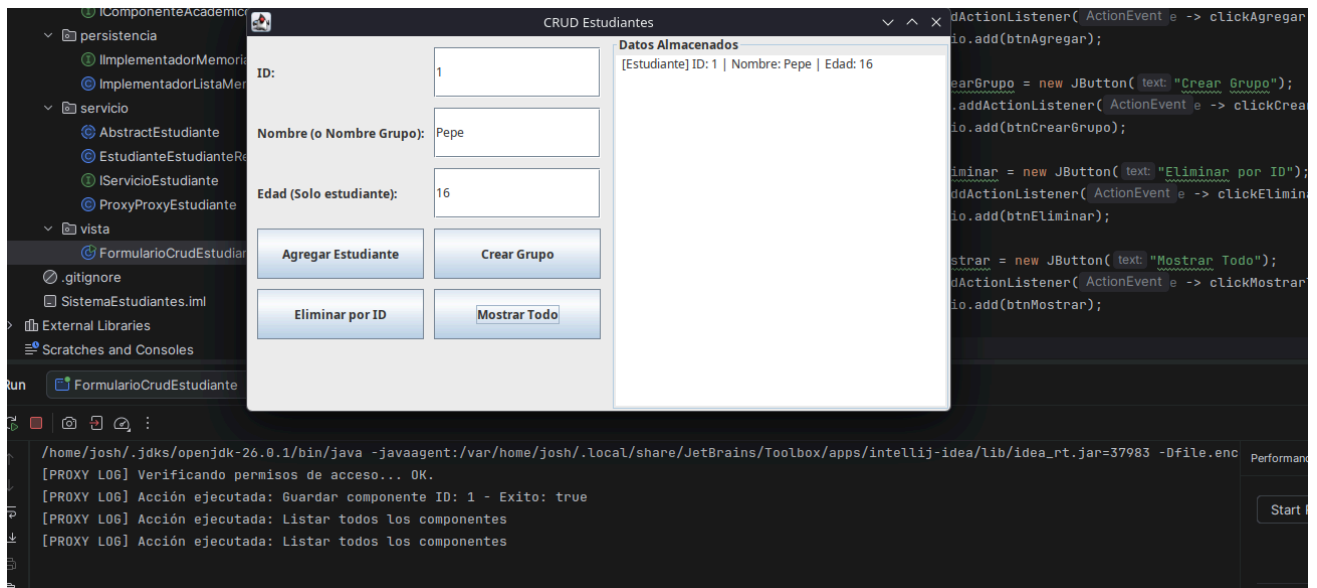
@enduml



Estructura del programa:



Ejecución programa:



CONCLUSIONES

La implementación conjunta de los patrones de diseño en este programa ha demostrado cómo una arquitectura bien planificada puede transformar un sistema rígido en uno altamente flexible y escalable. Al utilizar el patrón Composite, logramos unificar el tratamiento de estudiantes individuales y grupos bajo una misma interfaz común (IComponenteAcademico). Esto nos sirvió para simplificar enormemente el código del sistema, ya que el Controlador y el Repositorio pueden ejecutar acciones como mostrar datos o guardar información sin preocuparse de si están interactuando con un solo estudiante o con un árbol complejo que contiene múltiples subgrupos. Por su parte, el patrón Bridge nos permitió separar de forma limpia las reglas de negocio (la validación de si un ID ya existe) de los detalles técnicos del almacenamiento en memoria, previniendo el acoplamiento y facilitando el mantenimiento.

Además, la integración del patrón Proxy añadió una capa de seguridad y auditoría transparente para el resto del sistema, sin necesidad de modificar el código de la lógica principal. En nuestro programa, el Proxy sirvió como un "guardia de seguridad" que intercepta las peticiones del controlador, registra un log de actividades en la consola y verifica permisos antes de permitir que el servicio real guarde o elimine un registro. En conjunto, la combinación de estos tres patrones nos sirvió para cumplir con el Principio de Responsabilidad Única (cada clase tiene una única tarea definida), logrando que el programa, aunque no use base de datos, simule un entorno empresarial robusto, modular y preparado para el cambio.

RECOMENDACIONES

Implementar un sistema real de roles utilizando el Proxy:

Actualmente, el método `verificarAcceso()` dentro de `ProxyProxyEstudiante` es una simulación que siempre retorna verdadero. Se recomienda extender esta funcionalidad pasando el rol del usuario actual (ej. "Admin" o "Invitado") al constructor del Proxy. De esta forma, puedes configurar el Proxy para que bloquee métodos críticos como `borrar()` si el usuario es solo un invitado, aprovechando el verdadero potencial de seguridad que ofrece este patrón.

Aprovechar el Bridge para conectar una base de datos real:

Dado que el patrón Bridge ya estableció la interfaz `ImplementadorMemoria` (la cual abstrae la persistencia de datos), la recomendación a futuro es crear una nueva clase llamada, por ejemplo, `ImplementadorMySQL` o `ImplementadorMongoDB` que cumpla con esa interfaz. Gracias a la arquitectura que construimos, podrás pasar de usar listas en memoria a guardar datos en la nube simplemente cambiando una sola línea de código en tu método `main`, sin tocar el Controlador, el Proxy o las clases del Composite.

Sangolquí, a 09 de abril de 2026